



Data Engineering Course

What this deck builds

PageRank from the beginning: graphs, PageRank vocabulary, the first iterative algorithm, a numerical example, the Markov-chain view, damping, and a scalable MapReduce implementation.

- 1 Motivation
- 2 Graphs
- 3 PageRank Basics
- 4 Numerical Example
- 5 Markov Chains
- 6 Damping
- 7 MapReduce
- 8 Wrap-Up

Search engines face a ranking problem. Suppose many pages contain the words “data engineering.” Which page should appear first?

Text matching alone is not enough:

- A page can repeat keywords many times but still be low quality.
- A page can be useful even if it does not contain the exact query wording.
- Some pages are important because other important pages point to them.

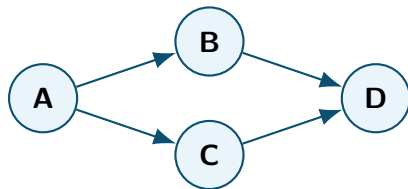
Core idea

A page is important if important pages link to it.

A graph represents objects and relationships:

- **Vertices** or **nodes**: the objects.
- **Edges**: the relationships.

$$G = (V, E)$$



A small directed graph

- In an **undirected graph**, an edge has no direction.
- In a **directed graph**, each edge has a direction.
- A web link is directed: if page A links to page B , it does not mean that page B links back to page A .

$$V = \{\text{pages}\}, \quad E = \{\text{hyperlinks}\}$$

For a page p :

- **Outlinks**: pages that p links to.
- **Inlinks**: pages that link to p .
- **Out-degree**: the number of outlinks from p .
- **In-degree**: the number of inlinks into p .

$$\Gamma(p) = \{q : p \rightarrow q\}, \quad |\Gamma(p)| = \text{outdeg}(p)$$

Important intuition

PageRank is not just counting inlinks. A link from a strong page contributes more, and a page that links to many pages divides its rank among them.

The PageRank score of a page p , written $\text{PR}(p)$, represents the page's importance in the link graph.

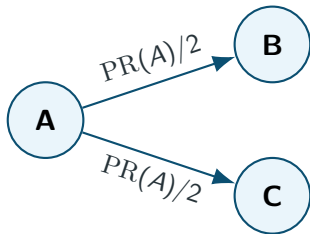
Ranks are usually normalized:

$$\sum_{p \in V} \text{PR}(p) = 1$$

This lets us interpret PageRank as a probability distribution over pages.

Each page distributes its current rank equally among its outlinks.

if $PR(A) = 0.30$ and $\text{outdeg}(A) = 3$, each outlink receives $\frac{0.30}{3} = 0.10$



A splits rank equally across two outlinks.

Initialization

$$\text{PR}_0(p) = \frac{1}{N}$$

Every page starts with equal rank.

Stopping condition

$$\sum_{p \in V} |\text{PR}_{t+1}(p) - \text{PR}_t(p)| < \varepsilon$$

Stop when scores change only slightly.

Start with a directed graph where every page has at least one outlink.

$$\text{PR}_{t+1}(p) = \sum_{q \rightarrow p} \frac{\text{PR}_t(q)}{|\Gamma(q)|}$$

Read the formula

The new rank of page p is the sum of all rank contributions sent by pages that link to p .

```
initialize  $PR[p] = 1 / N$  for every page  $p$ 

repeat until convergence:
    newPR[p] = 0 for every page  $p$ 

    for each page  $q$ :
        contribution =  $PR[q] / \text{number\_of\_outlinks}(q)$ 
        for each outlink  $p$  of  $q$ :
            newPR[p] += contribution

PR = newPR
```

Each iteration has two conceptual phases:

- ① **Distribute:** every page sends rank mass to its out-neighbors.
- ② **Collect:** every page sums the rank mass it receives from in-neighbors.

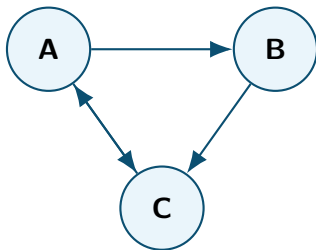
Data-engineering view

This is a repeated message-passing computation over graph edges.

Consider three pages:

$$V = \{A, B, C\}$$

$$A \rightarrow B, \quad A \rightarrow C, \quad B \rightarrow C, \quad C \rightarrow A$$



$$\Gamma(A) = \{B, C\}, \quad \Gamma(B) = \{C\}, \quad \Gamma(C) = \{A\}$$

$$|\Gamma(A)| = 2, \quad |\Gamma(B)| = 1, \quad |\Gamma(C)| = 1$$

Initialization:

$$\text{PR}_0(A) = \text{PR}_0(B) = \text{PR}_0(C) = \frac{1}{3}$$

Page A splits its rank between B and C :

$$A \rightarrow B : \frac{1/3}{2} = \frac{1}{6}, \quad A \rightarrow C : \frac{1/3}{2} = \frac{1}{6}$$

Page B sends all its rank to C :

$$B \rightarrow C : \frac{1/3}{1} = \frac{1}{3}$$

Page C sends all its rank to A :

$$C \rightarrow A : \frac{1/3}{1} = \frac{1}{3}$$

$$\text{PR}_1(A) = \frac{1}{3}, \quad \text{PR}_1(B) = \frac{1}{6}, \quad \text{PR}_1(C) = \frac{1}{6} + \frac{1}{3} = \frac{1}{2}$$

Page	PR_0	Incoming contributions	PR_1
A	$1/3$	from C: $1/3$	$1/3 = 0.3333$
B	$1/3$	from A: $1/6$	$1/6 = 0.1667$
C	$1/3$	from A: $1/6$, from B: $1/3$	$1/2 = 0.5000$

Use $\text{PR}_1(A) = 1/3$, $\text{PR}_1(B) = 1/6$, and $\text{PR}_1(C) = 1/2$.

$$A \rightarrow B : \frac{1}{6}, \quad A \rightarrow C : \frac{1}{6}, \quad B \rightarrow C : \frac{1}{6}, \quad C \rightarrow A : \frac{1}{2}$$

$$\text{PR}_2(A) = \frac{1}{2}, \quad \text{PR}_2(B) = \frac{1}{6}, \quad \text{PR}_2(C) = \frac{1}{3}$$

- After the first iteration, page C becomes strong because both A and B link to it.
- After the second iteration, page A becomes strong because the strong page C links to it.
- The algorithm keeps moving rank through the graph until the scores stabilize.

Imagine a user randomly surfing the web:

- ① The user is currently on some page.
- ② The user chooses one outgoing link uniformly at random.
- ③ The user moves to the linked page.
- ④ This repeats many times.

Interpretation

The PageRank score of a page is the long-run probability that the random surfer is on that page.

A Markov chain is a probabilistic process where the next state depends only on the current state, not on the full history.

For PageRank:

- States are pages.
- Transitions are hyperlinks.
- Transition probabilities are usually uniform over outlinks.

For the graph $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow C$, $C \rightarrow A$:

$$M_{ij} = \begin{cases} \frac{1}{\text{outdeg}(j)} & \text{if page } j \text{ links to page } i, \\ 0 & \text{otherwise} \end{cases}$$

$$M = \begin{bmatrix} 0 & 0 & 1 \\ \frac{1}{2} & 0 & 0 \\ \frac{1}{2} & 1 & 0 \end{bmatrix}$$

The PageRank update can be written as:

$$\mathbf{r}_{t+1} = M\mathbf{r}_t$$

At convergence:

$$\mathbf{r} = M\mathbf{r}$$

Stationary distribution

$\mathbf{r}$ is a stationary distribution of the Markov chain and an eigenvector of M with eigenvalue 1.

PageRank has two equivalent views:

- an iterative graph algorithm that sends messages along edges;
- repeated sparse matrix-vector multiplication.

Why this matters

Both views lead naturally to distributed implementations.

Dangling nodes

A dangling node has no outlinks:

$$D \rightarrow \emptyset$$

Its rank disappears unless handled.

Spider traps

A group of pages can point only to itself.
Rank can get stuck permanently inside the group.

Damping modifies the random surfer model:

- With probability d , the surfer follows an outgoing link.
- With probability $1 - d$, the surfer teleports to a random page.

$$d = 0.85$$

The surfer follows links 85% of the time and teleports 15% of the time.

- Prevents rank from getting stuck permanently in spider traps.
- Gives dangling-node rank a principled way to re-enter the graph.
- Makes every page reachable with positive probability.
- Helps guarantee a unique stable PageRank vector.

Let N be the number of pages, d the damping factor, and M_t the total rank mass of dangling nodes at iteration t .

$$\text{PR}_{t+1}(p) = \frac{1-d}{N} + d \left(\sum_{q \rightarrow p} \frac{\text{PR}_t(q)}{|\Gamma(q)|} + \frac{M_t}{N} \right)$$

- ① $\frac{1-d}{N}$: teleportation mass.
- ② Link-based contributions from in-neighbors.
- ③ $\frac{M_t}{N}$: redistributed dangling-node mass.

$$A \rightarrow B, \quad A \rightarrow C, \quad B \rightarrow C, \quad C \rightarrow A, \quad D \rightarrow \emptyset$$

$$N = 4, \quad \text{PR}_0(A) = \text{PR}_0(B) = \text{PR}_0(C) = \text{PR}_0(D) = 0.25, \quad d = 0.85$$

$$M_0 = \text{PR}_0(D) = 0.25, \quad \frac{M_0}{N} = 0.0625, \quad \frac{1-d}{N} = 0.0375$$

Incoming link contributions:

$$\text{in}(A) = 0.25, \quad \text{in}(B) = 0.125, \quad \text{in}(C) = 0.375, \quad \text{in}(D) = 0$$

$$\text{PR}_1(p) = 0.0375 + 0.85 (\text{in}(p) + 0.0625)$$

Page	Link contribution	Formula	PR_1
A	0.2500	$0.0375 + 0.85(0.3125)$	0.3031
B	0.1250	$0.0375 + 0.85(0.1875)$	0.1969
C	0.3750	$0.0375 + 0.85(0.4375)$	0.4094
D	0.0000	$0.0375 + 0.85(0.0625)$	0.0906

The web graph is huge:

- billions of pages;
- many billions of links;
- sparse but too large for one machine;
- iterative computation: the graph is processed many times.

This makes PageRank a natural example for MapReduce and other distributed data systems.

Each page can be represented as:

(page, current rank, adjacency list)

Page	Current rank	Adjacency list
A	0.25	[B, C]
B	0.25	[C]
C	0.25	[A]
D	0.25	[]

- ① **Map:** each page emits rank contributions to its outlinks.
- ② **Map:** each page also emits its adjacency list so the graph structure is preserved.
- ③ **Shuffle:** contributions are grouped by destination page.
- ④ **Reduce:** each page sums incoming contributions and applies damping.
- ⑤ **Repeat:** run another iteration using the new ranks.

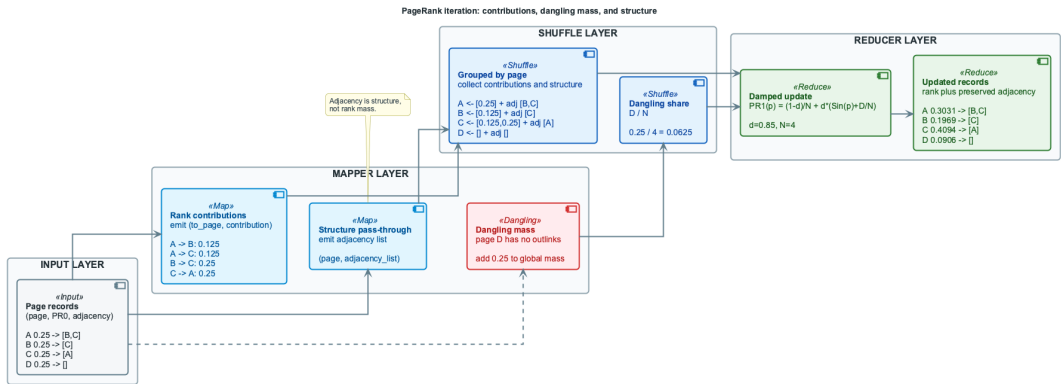


Figure: One PageRank iteration as a MapReduce dataflow.

Input:

$$(p, \text{PR}_t(p), \Gamma(p))$$

If p has outlinks:

$$\text{for each } q \in \Gamma(p), \quad \text{emit}(q, \text{PR}_t(p)/|\Gamma(p)|)$$

Also preserve graph structure:

$$\text{emit}(p, \text{AdjacencyList}(\Gamma(p)))$$

If p is dangling, add $\text{PR}_t(p)$ to global dangling mass.

Input for a page p :

$$(p, [\text{values grouped by key } p])$$

The values contain numeric contributions from in-neighbors and one adjacency list.

$$\text{sum_in}(p) = \sum \text{numeric contributions received by } p$$

$$\text{PR}_{t+1}(p) = \frac{1-d}{N} + d \left(\text{sum_in}(p) + \frac{M_t}{N} \right)$$

The reducer emits $(p, \text{PR}_{t+1}(p), \Gamma(p))$.

```
map(page p, rank r, adjacency list L):  
    emit(p, STRUCTURE(L))  
  
    if L is empty:  
        add r to DANGLING_MASS  
    else:  
        c = r / length(L)  
        for each destination q in L:  
            emit(q, CONTRIBUTION(c))  
  
combine(page p, values):  
    sum numeric CONTRIBUTION values  
    pass STRUCTURE values unchanged
```

```
reduce(page p, values):  
    L = adjacency list from STRUCTURE value  
    s = sum all CONTRIBUTION values  
    newRank = (1 - d) / N + d * (s + DANGLING_MASS / N)  
    emit(p, newRank, L)
```

Page	Rank	Outlinks
A	0.25	[B, C]
B	0.25	[C]
C	0.25	[A]
D	0.25	[]

Source page	Emitted key	Emitted value
A	B	contribution 0.125
A	C	contribution 0.125
A	A	structure [B, C]
B	C	contribution 0.250
B	B	structure [C]
C	A	contribution 0.250
C	C	structure [A]
D	D	structure []
D	global	dangling mass 0.250

After shuffle

Key	Values
A	contribution 0.250, structure [B, C]
B	contribution 0.125, structure [C]
C	contribution 0.125, contribution 0.250, structure [A]
D	structure []

Reducer output

Page	New rank	Structure
A	0.3031	[B, C]
B	0.1969	[C]
C	0.4094	[A]
D	0.0906	[]

- **Combiner**: safe for numeric contribution sums; must preserve adjacency lists.
- **Dangling mass**: track with a counter, special key, or separate aggregation job.
- **Convergence**: reducers can emit $|\text{PR}_{t+1}(p) - \text{PR}_t(p)|$.
- **Cost**: each iteration scans the graph and shuffles $O(|E|)$ data.
- **Skew**: hot pages may need two-stage aggregation.

- A web graph is a directed graph: pages are nodes, links are edges.
- Basic PageRank repeatedly sends each page's rank through its outgoing links.
- PageRank is also a Markov chain: ranks are long-run visit probabilities.
- Damping adds teleportation, handles dangling nodes, and prevents rank traps.
- MapReduce implements each iteration as contribution emission, shuffle, and reduction by summation plus damping.

- ➊ Given $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$, and $D \rightarrow C$, compute one simple PageRank iteration from equal initialization.
- ➋ For the same graph, write the transition matrix M .
- ➌ Add damping with $d = 0.85$. If there are no dangling nodes, what changes?
- ➍ Explain why the adjacency list must pass through mapper and reducer.
- ➎ Suppose one page receives 40% of all graph links. What MapReduce bottleneck might appear, and how could you reduce it?